BƯỚC 33 - GRAPHQL VARIABLES

Giáo án GraphQL với Ruby on Rails - nội dung 1:1

Bước 33 là kiến thức của GraphQL, không phải của Rails.

Rails chỉ là nơi nhận `variables` và truyền chúng vào `graphql-ruby`. Trong giáo án, mục tiêu của Bước 33 là "tách query document khỏi dữ liệu runtime", tức là phần này nằm ở GraphQL document / client request.

Bước 33 — GraphQL Variables

Hãy hiểu theo sơ đồ này:

```
Frontend
   │
   │ g□i
   ▼
GraphQL document
+
variables
   │
   ▼
POST /graphql
   │
   ▼
Rails GraphqlController
   │
   ▼
graphql-ruby
   │
   ▼
Resolver / Mutation
```

Trong đó:

```
Variables
→ tính năng c□a GraphQL

GraphqlController
→ Rails ch□ nhận và chuy□n ti□p variables

Resolver
→ nhận giá trị sau khi GraphQL đã parse/validate
```

1. Trước Bước 33: hard-code dữ liệu

Ví dụ bạn muốn lấy Project có ID 10:

```
query {
  project(id: 10) {
    id
    name
  }
}
```

Ở đây:

```
10
```

bị viết chết trong GraphQL document.

Muốn Project 20 thì lại phải tạo:

```
query {
```

```
  project(id: 20) {
    id
    name
  }
}
```

Không nên làm như vậy trong frontend thực tế.

2. Dùng Variable

Ta đổi thành:

```
query GetProject($id: ID!) {
  project(id: $id) {
    id
    name
  }
}
```

Và dữ liệu được gửi riêng:

```
{
  "id": "10"
}
```

Lúc khác:

```
{
  "id": "20"
}
```

GraphQL document không đổi.

Chỉ `variables` thay đổi.

Đây chính là ý nghĩa:

```
GraphQL document
→ c□u trúc c□ định

Variables
→ dữ liệu runtime
```

3. `$id: ID!` nghĩa là gì?

Phân tích:

```
$id: ID!
```

Có 3 phần:

```
$id
│
└── tên variable

ID
│
└── GraphQL type

!
│
```

```
└── required / không được null
```

Tức:

```
query GetProject($id: ID!) {
```

nói với GraphQL:

Query này cần một variable tên `id`, kiểu `ID`, và bắt buộc phải có.

4. `$id` được dùng ở đâu?

Ở đây:

```
project(id: $id)
```

Ta có:

```
$id
↓
được truy□n vào argument id
↓
project(id: ...)
```

Sau đó Rails resolver của bạn:

```
def resolve(id:)
  ...
end
```

nhận giá trị đó.

Flow đầy đủ:

```
variables
{
  "id": "10"
}

     ↓

$id

     ↓

project(id: $id)

     ↓

ProjectResolver

     ↓

resolve(id: "10")
```

5. Rails có phải tạo `$id` không?

Không.

Bạn không viết trong Rails:

```
$id = ...
```

Không có chuyện đó.

`$id` chỉ tồn tại trong GraphQL document.

Rails resolver chỉ nhận:

```
def resolve(id:)
```

Ví dụ:

```
module Resolvers
  class ProjectResolver < BaseResolver
    type Types::ProjectType,
         null: false

    argument :id,
             ID,
             required: true

    def resolve(id:)
      authenticate_user!

      project =
        find_project!(id)

      authorize_read!(project)
    end
  end
end
```

Bạn không sửa Resolver chỉ vì bắt đầu dùng Variables.

6. Vậy Rails làm gì?

Rails chỉ nhận HTTP request.

Frontend gửi:

```
{
  "query": "query GetProject($id: ID!) { project(id: $id) { id name } }",
  "variables": {
    "id": "10"
  }
}
```

`GraphqlController` nhận:

```
params[:query]
params[:variables]
```

Sau đó:

```
result =
  YourAppSchema.execute(
    params[:query],
    variables: params[:variables],
    context: {
      current_user: current_user
    }
  )
```

Tức Rails làm:

```
HTTP JSON
   ↓
params
   ↓
Schema.execute
```

Còn GraphQL làm:

```
$id: ID!
↓
validate type
↓
gán variable
↓
resolver
```

7. Ví dụ với Axios

Đây sẽ làm rõ nhất.

Frontend:

```
const response = await axios.post(
  "http://localhost:3000/graphql",
  {
    query: `
      query GetProject($id: ID!) {
        project(id: $id) {
          id
          name
        }
      }
    `,

    variables: {
      id: "10"
    }
  },
  {
    withCredentials: true
  }
);
```

Có hai phần hoàn toàn riêng biệt:

```
query: `...`
```

và:

```
variables: {
  id: "10"
}
```

8. Khi bấm button thì variable thay đổi

Ví dụ React:

```
const loadProject = async (projectId) => {
  const response = await axios.post(
    "/graphql",
    {
```

```
      query: `
        query GetProject($id: ID!) {
          project(id: $id) {
            id
            name
          }
        }
      `,
      variables: {
        id: projectId
      }
    },
    {
      withCredentials: true
    }
  );

  console.log(
    response.data.data.project
  );
};
```

Button:

```
<button
  onClick={() => loadProject("10")}
>
  Load Project 10
</button>
```

Button khác:

```
<button
  onClick={() => loadProject("20")}
>
  Load Project 20
</button>
```

Query vẫn là một query:

```
query GetProject($id: ID!) {
  project(id: $id) {
    id
    name
  }
}
```

Chỉ variable thay đổi:

```
Button 1
→ id = 10

Button 2
→ id = 20
```

Đây chính là mục đích của Variables.

9. Tại sao không nối String?

Không nên:

```
const query = `
  query {
    project(id: ${projectId}) {
      id
```

```
      name
    }
  }
`;
```

Bạn đang tự build GraphQL document mỗi lần.

Nên:

```
const query = `
  query GetProject($id: ID!) {
    project(id: $id) {
      id
      name
    }
  }
`;
```

và:

```
const variables = {
  id: projectId
};
```

10. Mutation cũng giống hệt

Ví dụ `UpdateTask`.

Không nên:

```
mutation {
  updateTask(
    input: {
      id: 100
      attributes: {
        status: IN_PROGRESS
      }
    }
  ) {
    task {
      id
      status
    }
  }
}
```

Nên:

```
mutation UpdateTask(
  $id: ID!
  $attributes: TaskUpdateInput!
) {
  updateTask(
    input: {
      id: $id
      attributes: $attributes
    }
  ) {
    task {
      id
      title
      status
      priority
    }

    errors
```

```
  }
}
```

Variables:

```
{
  "id": "100",
  "attributes": {
    "status": "IN_PROGRESS",
    "priority": "HIGH"
  }
}
```

11. Cách tốt hơn với mutation hiện tại của bạn

Do Bước 19 bạn đã có:

```
TaskUpdateInput
```

nên frontend có thể gửi cả input object.

Ví dụ:

```
mutation UpdateTask(
  $input: UpdateTaskInput!
) {
  updateTask(input: $input) {
    task {
      id
      title
      status
      priority
    }

    errors
  }
}
```

Variables:

```
{
  "input": {
    "id": "100",
    "attributes": {
      "status": "IN_PROGRESS",
      "priority": "HIGH"
    }
  }
}
```

Tôi thích cách này hơn vì frontend chỉ phải quản lý:

```
$input
```

12. Filter của Bước 28 cũng dùng Variable

Ta vừa xây:

```
TaskFilterInput
```

Frontend không nên:

```
tasks(
  projectId: 10
  filter: {
    status: [TODO]
    priority: [HIGH]
  }
)
```

Mà nên:

```
query GetTasks(
  $projectId: ID!
  $filter: TaskFilterInput
) {
  tasks(
    projectId: $projectId
    filter: $filter
    first: 20
  ) {
    nodes {
      id
      title
      status
      priority
    }
  }
}
```

Variables:

```
{
  "projectId": "10",
  "filter": {
    "status": [
      "TODO",
      "IN_PROGRESS"
    ],
    "priority": [
      "HIGH"
    ]
  }
}
```

13. GraphQL Enum trong Variables

Trong GraphQL document:

```
status: TODO
```

Nhưng trong JSON variables:

```
{
  "status": "TODO"
}
```

Tại sao?

Vì JSON chỉ có:

```
string
number
boolean
null
array
object
```

JSON không có type `GraphQL Enum`.

GraphQL nhận:

```
"TODO"
```

rồi convert theo:

```
TaskStatusEnum
```

thành Ruby value:

```
"todo"
```

Flow:

```
JSON
"TODO"

↓

GraphQL Enum

↓

TaskStatusEnum

↓

Ruby
"todo"
```

14. Pagination cũng phải dùng Variables

Không:

```
tasks(
  first: 20
  after: "ABC123"
)
```

Nên:

```
query GetTasks(
  $projectId: ID!
  $first: Int!
  $after: String
) {
  tasks(
    projectId: $projectId
    first: $first
    after: $after
  ) {
    nodes {
      id
      title
    }

    pageInfo {
      hasNextPage
      endCursor
    }
  }
}
```

Variables lần đầu:

```
{
  "projectId": "10",
  "first": 20,
  "after": null
}
```

Server trả:

```
{
  "hasNextPage": true,
  "endCursor": "ABC123"
}
```

Lần sau:

```
{
  "projectId": "10",
  "first": 20,
  "after": "ABC123"
}
```

Query không thay đổi.

Chỉ `variables.after` thay đổi.

15. Required và Optional Variable

Ví dụ:

```
query GetTasks(
  $projectId: ID!
  $filter: TaskFilterInput
  $first: Int! = 20
  $after: String
) {
```

Có nghĩa:

```
$projectId: ID!
→ b□t buộc

$filter: TaskFilterInput
→ optional

$first: Int! = 20
→ có default 20

$after: String
→ optional
```

16. Nếu gửi sai type thì sao?

Query:

```
query GetTasks($first: Int!) {
```

Bạn gửi:

```
{
```

```
  "first": "hello"
}
```

GraphQL reject trước khi Resolver chạy.

Resolver thậm chí chưa được gọi.

Flow:

```
variables
↓
GraphQL type validation
↓
SAI
↓
error

Resolver
✗ không chạy
```

Đây là lợi ích rất lớn.

17. RSpec cũng đang dùng Variables

Bước 30 bạn đã viết:

```
result =
  graphql_execute(
    query,
    variables: {
      id: project.id
    },
    context: {
      current_user: user
    }
  )
```

Đây chính là Bước 33.

Trong test:

```
variables: {
  id: project.id
}
```

tương đương frontend gửi:

```
{
  "variables": {
    "id": "10"
  }
}
```

18. Phân biệt cho thật rõ

Đây là phần bạn nên nhớ nhất:

Thành phần	Thuộc về
`$id`	GraphQL

Thành phần	Thuộc về
`$filter`	GraphQL
`$input`	GraphQL
`ID!`	GraphQL
`TaskFilterInput`	GraphQL Schema
`variables: {...}`	GraphQL request/client
`params[:variables]`	Rails Controller
`Schema.execute(variables:)`	graphql-ruby trong Rails
`resolve(id:)`	Rails/graphql-ruby Resolver
`axios.post()`	Frontend

19. Flow hoàn chỉnh với Project của bạn

Ví dụ frontend:

```
axios.post(
  "/graphql",
  {
    query: `
      query GetTasks(
        $projectId: ID!
        $filter: TaskFilterInput
        $first: Int!
      ) {
        tasks(
          projectId: $projectId
          filter: $filter
          first: $first
        ) {
          nodes {
            id
            title
          }
        }
      }
    `,

    variables: {
      projectId: "10",
      filter: {
        status: ["TODO"]
      },
      first: 20
    }
  }
);
```

Rails:

```
params[:query]
params[:variables]
```

↓

```
YourAppSchema.execute(
  params[:query],
  variables: params[:variables],
  context: {
```

```
    current_user: current_user
  }
)
```

GraphQL:

```
$projectId → "10"

$filter
→ {
    status: ["TODO"]
  }

$first → 20
```

↓

Resolver:

```
def resolve(
  project_id:,
  filter: nil,
  sort_by:,
  sort_direction:
)
```

nhận:

```
project_id
# "10"

filter
# TaskFilterInput...
```

rồi ActiveRecord mới chạy.

Kết luận Bước 33

Variables thuộc GraphQL.

Rails chỉ hỗ trợ GraphQL bằng `graphql-ruby` và nhận chúng qua HTTP request.

Hãy nhớ:

```
Frontend tạo variable
        ↓
GraphQL định nghĩa type cho variable
        ↓
Rails Controller nhận variable
        ↓
graphql-ruby validate/convert
        ↓
Resolver nhận Ruby value
```

Và mục tiêu cuối cùng của Bước 33 là:

```
KHÔNG:

project(id: 10)

KHÔNG:

"project(id: #{projectId})"
```

```
MÀ:

project(id: $id)

+

variables: {
  id: projectId
}
```

Từ Bước 33 trở đi, mọi Query/Mutation frontend nên mặc định dùng Variables.

Ghi chú bổ sung sau Bước 33

Đúng. Ở Bước 33 — GraphQL Variables, về cơ bản bạn không cần sửa business code Rails hiện tại.

Nếu `GraphqlController` của bạn đã truyền `params[:variables]` vào schema, ví dụ:

```
result = YourAppSchema.execute(
  params[:query],
  variables: params[:variables],
  context: {
    current_user: current_user
  },
  operation_name: params[:operationName]
)
```

thì Rails đã sẵn sàng.

Bạn không cần sửa:

```
ProjectResolver
TasksResolver
CreateTask
UpdateTask
ProjectPolicy
TaskPolicy
Service Objects
Models
```

Có một chỗ duy nhất bạn nên kiểm tra ở Rails là:

```
app/controllers/graphql_controller.rb
```

phải có:

```
variables: variables
```

hoặc tương đương:

```
variables: params[:variables]
```

trong:

```
YourAppSchema.execute(...)
```

Nếu đã có thì Bước 33 không cần sửa code Rails nữa.

Có thể ghi ngắn gọn vào giáo án:

Bước 33 = chú ý phía GraphQL Client / Frontend.

Rails:
→ chỉ kiểm tra GraphqlController đã forward variables.

Resolver/Mutation/Policy/Service:
→ không cần thay đổi.